CAL POLY

CSC 369 - Introduction to Distributed Computing

Spam Email Classification Using BoW, TF-IDF, and KNN

Arturo Ordaz-Gutiérrez

Overview & Dataset

Overview

- Classifies emails as Spam or Ham using two feature extraction methods compared through the same classifier
- All algorithms implemented from scratch using only Spark RDDs

Source

- Kaggle Spam Emails Dataset: 193,852 emails
- **Distribution:** 91,000 spam, 102,000 ham
- Raw CSV spans 889,000 lines because email bodies contain newlines, commas, and quotes inside quoted fields
- Dataset is large enough to demonstrate the value of distributed computing, KNN compares every test email against every training email

Preprocessing

- Custom multiline CSV parser tracks quoted fields across multiple lines to produce one (label, text) pair per email
- Text cleaning: convert to lowercase, remove non-alphabetic characters, filter ~90 stopwords and HTML artifacts (img, href, nbsp)
- Words shorter than 3 characters are removed Dataset is shuffled and split 80/20 into train and test RDDs using sc.parallelize(), both persisted with persist()

Bag of Words

- Each email is represented as a map of word → count
- Implementation uses the MapReduce word count pattern from class:
  - flatMap each email into ((docId, word), 1.0) pairs
  - reduceByKey(_ + _) sums counts across the cluster
  - groupByKey reassembles per-document word count maps
- join with email RDD attaches the Spam/Ham label to each vector

A small example showing one email going through the pipeline:

Input: "buy cheap deals now"

Output: {buy: 1.0, cheap: 1.0, deals: 1.0}

TF-IDF

- Problem with BoW: all words weighted equally. Common words mislead the classifier
- TF-IDF weights words by importance:
  - TF (Term Frequency) = word count / max count in that document
  - DF (Document Frequency) = how many documents contain this word
  - IDF (Inverse Document Frequency) = log(N / DF)
  - TF-IDF = TF × IDF

Words appearing everywhere/common words: low IDF = low weight = not important

Words unique to spam or ham/uncommon words: high IDF = high weight = very important

TF-IDF Implementation

1. TF: Same flatMap + reduceByKey as BoW, then normalize by max count per document
2. DF: flatMap unique words per doc into (word, 1.0), reduceByKey to sum, collectAsMap to bring to driver
3. TF-IDF: Compute IDF = log(N / DF), distribute IDF map to all workers using broadcast, then map to multiply TF × IDF

Broadcast is used because the IDF map is a small lookup table every worker needs, shipping it once per node is far more efficient than shipping it with every task

KNN with Cosine Distance

- For each test email: compute similarity to every training email, pick k most similar, majority vote on their labels
- Cosine similarity = dot(v1, v2) / (||v1|| × ||v2||)
  - Range: 0 (no words in common) to 1 (identical distribution)
  - Only words in both vectors contribute, efficient for sparse text
- Implementation:
  - Training vectors collected and distributed via broadcast
  - Test RDD processed with mapPartitions, train array loaded once per partition, reused for all test emails in that partition
  - For each test email: compute similarity to all train emails -> sortBy descending -> take(k) -> groupBy label -> majority vote

Results (k = 5, 8,000 emails)

	Accuracy	Precision	Recall	F1
Bag of Words	0.7288	0.7161	0.6851	0.7003
TF-IDF	0.8975	0.9034	0.8716	0.8872

	True Positive	False Positive	True Negative	False Negative
Bag of Words	507	201	659	223
TF-IDF	645	69	791	95

Key takeaway:

- TF-IDF achieves 89.75% accuracy vs BoW's 72.88%
- TF-IDF produces only 69 false positives vs BoW's 201

Why TF-IDF Won?

- BoW treats all words equally, common words that appear in both spam and ham can mislead KNN
- TF-IDF down-weights common words and up-weights distinctive words
- Result: TF-IDF's precision jumps from 0.72 to 0.90, nearly 3x fewer false positives (69 vs 201)
- In a real email system, falsely marking ham as spam is more disruptive than letting some spam through, TF-IDF is clearly better for this task